PotensicProxy_TAF – GitHub-Buildfehler Debug-Signierung

Projekt: PotensicProxy_TAF | Version: v0.9845.7-rc6 | Datum: 2026-10-14

Status: Release-Kandidat, APK-Build offen | Technische Bearbeitung: ChatGPT

Ablage: DOCUMENTATION / Auslieferung | Grundlage: logs_104436650670.zip

Befund

Der GitHub-Lauf bricht im Schritt Prepare stable debug signing ab: TAF_DEBUG_KEYSTORE_B64 fehlt. Die vier Secrets wurden leer übergeben. Vorherige npm- und Quellprüfungen liefen weiter; ein APK-Build fand nicht statt.

Ursache

rc5 macht die stabile Debug-Signierung bereits für jeden BL_-Test-Build zur zwingenden Voraussetzung, obwohl der übliche Android-Debug-Build auch ohne projektspezifischen Keystore möglich ist.

Korrektur

Ein neuer Prüfschritt erkennt vollständige Signier-Secrets. Fehlen sie, wird eine Warnung ausgegeben und die gewöhnliche Debug-APK als Artifact gebaut; GitHub-Prerelease-Veröffentlichung wird dann übersprungen. Bei vollständigen Secrets wird der bereitgestellte Keystore verwendet und der In-App-Update-Releasepfad bleibt aktiv.

Einrichtung

GitHub → Settings → Secrets and variables → Actions: TAF_DEBUG_KEYSTORE_B64, TAF_DEBUG_STORE_PASSWORD, TAF_DEBUG_KEY_ALIAS, TAF_DEBUG_KEY_PASSWORD. Alle vier gehören zu EINEM langfristig gesicherten Schlüssel. Ohne passenden Schlüssel ist ein Update einer bereits anders signierten APK nicht möglich.

Warnungen

npm audit meldete Schwachstellen (Bericht als GitHub-Artefakt); aus dem vorliegenden ZIP sind Schwere und Pakete nicht ersichtlich. GitHub-Actions-Node-Warnungen und punycode/url.parse-Hinweise verursachten nicht den Abbruch.

Sicherheit

Ein gewöhnlicher Debug-Build ohne stabile Signierung darf NICHT als kompatibles In-App-Update veröffentlicht werden. Die Android-Signaturprüfung bleibt unverändert.

Prüfungen

Workflow-YAML gelesen; beide Ausführungszweige und Bedingung statisch geprüft; ZIP-Struktur und Integrität geprüft. Vollständiger GitHub-APK-Build und Gerätetests sind noch offen.

Änderungshistorie

v0.9845.7-rc5 → v0.9845.7-rc6 (14.10.2026): Debug-Build ohne Secrets ermöglicht; sichere In-App-Update-Veröffentlichung bleibt an stabile Signierung gebunden.

Geänderte Dateien

.github/workflows/build-apk.yml

Abnahme (offen)

BL_ ohne Secrets: Artifact ja, Prerelease nein. BL_ mit Secrets: stabil signiertes Prerelease; anschließend Signatur-, Versions- und Geräteprüfung.